

La solution : une sortie structurée avec Pydantic

Imposer le format JSON avec **output_schema**

Référence : [documentation ADK — structuration des données](#)

D'après la documentation ADK, utilisez Pydantic BaseModel pour définir la structure exacte dont vous avez besoin :

```
from pydantic import BaseModel, Field

class ProductInfo(BaseModel):
    product_name: str = Field(description="The name of the product")
    price: float = Field(description="Le prix est en USD")
    storage: str = Field(description="The storage capacity")

structured_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="""Extrais les informations sur le produit et réponds au format JSON.
    Format: {"product_name": "name", "price": 999.99, "storage": "256GB"}""",
    output_schema=ProductInfo # Impose cette structure exacte
)
```

Extrait de la documentation ADK :

"output_schema (facultatif)" : définit un schéma représentant la structure de sortie souhaitée. S'il est défini, la réponse finale de l'agent *doit* être une chaîne JSON conforme à ce schéma."

Concepts fondamentaux

1. Définir des schémas avec Pydantic

Référence : [documentation ADK – exemple Python](#)

Créez un modèle Pydantic qui représente le résultat souhaité :

```
from pydantic import BaseModel, Field

class CapitalOutput(BaseModel):
    capital: str = Field(description="The capital of the country")
```

Points essentiels :

- **Doit hériter de **BaseModel**** : vous ne pouvez pas utiliser de dictionnaires Python ni de classes simples.
- Chaque champ possède un type (**str**, **float**, **int**, **bool**, etc.).

- Utilisez `Field(description=...)` pour aider le LLM à comprendre chaque champ.
- Transmettez la **classe elle-même** à `output_schema`, et non une instance :
`output_schema=CapitalOutput`

Important : D'après la documentation ADK, "les schémas d'entrée et de sortie sont généralement des classes `Pydantic BaseModel`". Vous devez définir votre schéma en tant que classe Pydantic. Les dictionnaires comme `{"name": "string"}` ne fonctionneront pas.

2. Utiliser `output_schema` dans les agents

Référence : [documentation ADK — structuration des données](#)

Appliquez le schéma à votre agent :

```
structured_agent = LlmAgent(  
    model="gemini-2.5-flash",  
    name="capital_finder",  
    instruction="""Tu es un agent d'informations sur les capitales.  
    Quand on te donne un pays, réponds UNIQUEMENT avec un objet JSON contenant la  
    capitale.  
    Format : {"capital": "capital_name"}""",  
    output_schema=CapitalOutput # Impose une sortie au format JSON  
)
```

Fonction :

- L'agent **doit** renvoyer un objet JSON correspondant au schéma.
 - Les réponses non valides sont automatiquement rejetées.
 - Votre code bénéficie d'une structure garantie.
-

3. Stocker les résultats avec `output_key`

Référence : [documentation ADK - output_key](#)

Enregistrez la réponse de l'agent dans l'état de session pour une utilisation ultérieure :

```
structured_agent = LlmAgent(  
    model="gemini-2.5-flash",  
    instruction="Extrais la capitale au format JSON",  
    output_schema=CapitalOutput,  
    output_key="found_capital" # Store in session.state["found_capital"]  
)
```

Extrait de la documentation ADK :

"output_key (facultatif) : fournissez une clé de chaîne. Si cette clé est définie, le

contenu textuel de la *dernière* réponse de l'agent sera automatiquement enregistré dans le dictionnaire d'état de la session sous cette clé.

Cas d'utilisation :

- Transmettre des données entre les agents dans les workflows
- Stocker les informations extraites pour un traitement ultérieur
- Créer des pipelines multi-étapes

Remarque importante concernant les workflows multi-agent :

Bien que ce module affiche `output_schema` sur le `root_agent`, la sortie structurée est tout aussi utile pour les sous-agents intermédiaires dans les workflows multi-agents (comme abordé dans le cours 5). Lorsque plusieurs agents travaillent ensemble, l'utilisation de `output_schema` sur les sous-agents garantit un format de données cohérent lors du transfert d'informations entre les agents. Exemple :

Le sous-agent extrait des données structurées

```
extraction_agent = LlmAgent(  
    model="gemini-2.5-flash",  
    name="data_extractor",  
    output_schema=ProductInfo, # Sortie structurée pour le prochain agent  
    output_key="extracted_data"  
)
```

Un autre agent utilise ces données structurées

```
analysis_agent = LlmAgent(  
    model="gemini-2.5-flash",  
    name="data_analyzer",  
    # Reçoit des données structurées de l'extraction_agent via l'état de session  
)
```

Vous pouvez utiliser une sortie structurée à n'importe quel niveau d'agent (racine ou intermédiaire), selon l'endroit où vous avez besoin de formats de données garantis.

4. Schémas complexes

Référence : [documentation ADK — structuration des données](#)

Concevoir des structures plus sophistiquées :

```
from pydantic import BaseModel, Field  
from typing import List, Optional  
  
class ProductDetails(BaseModel):  
    name: str = Field(description="Nom du produit")  
    price: float = Field(description="Prix en USD")  
    storage_options: List[str] = Field(description="Capacités de stockage disponibles")  
    in_stock: bool = Field(description="Disponibilité du produit")
```

```

discount: Optional[float] = Field(default=None, description="Pourcentage de
réduction, le cas échéant")

product_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="""Extrais les informations complètes du produit au format JSON.
    À inclure : nom, prix, options de stockage (liste), disponibilité (valeur
    booléenne) et réduction (le cas échéant).""",
    output_schema=ProductDetails
)

```

Types compatibles :

- Basique : `str`, `int`, `float`, `bool`
- Collections : `List[T]`, `Dict[str, T]`
- Facultatif : `Optional[T]` pour les champs pouvant être nuls
- Imbriqué : autres classes `BaseModel`

Important : exhaustivité du schéma

Le schéma définit la structure de sortie EXACTE. Le LLM n'inclura QUE les champs que vous définissez dans votre classe Pydantic `BaseModel`. Si vous avez besoin d'objets imbriqués comme "metadata", "errors" ou "pagination" dans votre résultat, vous devez les définir explicitement dans le schéma :

```

from pydantic import BaseModel, Field
from typing import List, Dict

class ApiResponse(BaseModel):
    status: str = Field(description="succès, erreur ou partiel")
    status_code: int = Field(description="code d'état HTTP")
    data: Dict = Field(description="Données principales de la réponse")
    metadata: Dict = Field(description="Métadonnées de la requête") # Doit être
    défini pour apparaître
    errors: List[str] = Field(default=[], description="Messages d'erreur, le cas
    échéant")

# Sans "metadata" dans le schéma, le modèle ne l'inclura pas dans la sortie.
# Le schéma est un contrat : seuls les champs définis apparaissent dans les réponses

```

Vous obtenez ainsi exactement la structure dont vous avez besoin, ni plus, ni moins.

Exemple pratique

Créons un agent complet d'extraction de produits avec une sortie structurée.

Étape 1 : Créer le projet

```
adk create product_extractor
```

```
cd product_extractor
```

Étape 2 : Écrire l'agent avec une sortie structurée

Remplacez le contenu de `agent.py` par le code suivant :

```
"""
Agent d'extraction de produits avec une sortie JSON structurée.
Démonstration de output_schema d'ADK avec Pydantic BaseModel.
"""

from google.adk.agents import LlmAgent
from pydantic import BaseModel, Field

# Étape 1 : Définir la structure de sortie avec Pydantic
class ProductInfo(BaseModel):
    product_name: str = Field(description="Le nom complet du produit")
    price: float = Field(description="Le prix en USD")
    storage: str = Field(description="Capacité de stockage (exemple, 256 Go)")
    color: str = Field(default="Not specified", description="Couleur du produit, le cas échéant")

# Étape 2 : Créer l'agent avec output_schema
root_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="product_extractor",
    description="Extrais les informations produit des messages des utilisateurs et renvoie-les sous forme de données JSON semi-structurées",
    instruction="""Tu es un extracteur d'informations produit.

Ta tâche :
- Lire le message de l'utilisateur concernant un produit
- Extraire les éléments suivants : product_name, price, storage et color (le cas échéant)
- Répondre UNIQUEMENT avec des données JSON valides correspondant à ce format :

{
  "product_name" : "nom du produit ici",
  "prix" : 999,99,
  "stockage" : "256 Go",
  "couleur" : "noir cosmique"
}

Règles :
- Le prix doit être un nombre (sans le signe dollar).
- Le stockage doit inclure l'unité (Go, To).
- Si la couleur n'est pas mentionnée, indiquer "Non spécifié".
- Ne renvoyer QUE les données JSON, sans texte explicatif""",
    output_schema=ProductInfo, # Impose cette structure exacte
    output_key="extracted_product" # Stocke le résultat dans l'état de session
)
```

Étape 3 : Exécuter et tester

adk web

Accédez à <http://localhost:8000> et testez les entrées suivantes :

Test 1 : informations complètes

Vous : "Je veux l'iPhone 15 Pro avec 256 Go de stockage en noir cosmique pour 999 \$"

Résultat JSON attendu :

```
{
  "product_name": "iPhone 15 Pro",
  "price": 999.0,
  "storage": "256 Go",
  "color": "noir cosmique"
}
```

Test 2 : couleur manquante

Vous : "Le Samsung Galaxy S24 avec 512 Go de stockage, coûte 1 199 \$"

Résultat JSON attendu :

```
{
  "product_name": "Samsung Galaxy S24",
  "price": 1199.0,
  "storage": "512 Go",
  "color": "Non spécifié"
}
```

Test 3 : autre format

Vous : "Procure-moi un MacBook Pro avec 1 To de stockage, le prix est de 2 499 \$."

Résultat JSON attendu :

```
{
  "product_name": "MacBook Pro",
  "price": 2 499.0,
  "storage": "1 To",
  "color": "Non spécifié"
}
```

Points à noter :

-  La sortie est **toujours** un objet JSON valide
-  Même structure **à chaque fois**
-  Les champs manquants reçoivent des valeurs par défaut
-  Facile à analyser dans votre application

Points à retenir

- **output_schema** impose une sortie JSON structurée à l'aide de Pydantic BaseModel.
- **Transmettez la classe** (et non une instance) : **output_schema=ProductInfo**.
- **output_key** stocke les résultats dans l'état de session pour l'intégration des workflows.
- **Les instructions doivent guider le format JSON** : elles indiquent à l'agent ce qu'il doit générer.
- **La validation est automatique** : les réponses non valides sont rejetées.
- **Utilisez des descriptions de champ** pour aider le LLM à comprendre l'utilité de chaque champ.